

Week 2 Day 3: Image, Dockerfile, Build, Registry

Overview

Day 3는 Docker image를 직접 만들고 읽는 날이다. Day 1~2에서는 official image를 pull해서 실행했다. 이제 표준 실습 앱 코드를 제공하고, Dockerfile로 image를 build한 뒤 layer/cache/tag/digest/registry 관점에서 검증한다.

오늘의 핵심 질문은 다음과 같다.

내 코드와 실행 조건을 어떤 Dockerfile과 image tag로 포장해야 다른 사람이 같은 결과를 실행할 수 있는가?

Day 3도 CLI 실험 중심이다. build 명령, run 검증, image history/inspect, tag/digest 확인, failure drill을 모두 code block으로 제공한다. Docker Hub push는 기본 요구하지 않고, 선택 실습으로만 둔다. push를 한다면 credential, public repository, secret 포함 여부를 먼저 gate로 확인한다.

Learning Goals

- image, layer, tag, digest의 차이를 설명한다.
- Dockerfile의 FROM, WORKDIR, COPY, RUN, CMD, EXPOSE를 읽고 수정한다.
- build context와 .dockerignore가 image size와 secret risk에 미치는 영향을 확인한다.
- 제공된 표준 앱을 docker build로 image로 만들고 docker run으로 HTTP 확인한다.
- docker history, docker image inspect, image size, cache hit/miss를 evidence로 남긴다.
- Docker Hub official image와 custom image의 provenance를 구분한다.
- tag/push/pull 흐름을 설명하되, push는 credential/security gate 이후 선택으로 수행한다.
- missing file, wrong CMD, wrong port, bloated context 같은 build/runtime 실패를 RCA로 기록한다.

Lesson Index

- 1교시: image와 layer, tag, digest - pull한 image를 images, history, inspect로 읽기
- 2교시: Dockerfile 기본 문법 - FROM, WORKDIR, COPY, RUN, CMD, EXPOSE
- 3교시: build context와 .dockerignore - source tree, secret 제외, context size 확인
- 4교시: 표준 앱 image build/run - 제공 코드로 image build, container run, HTTP 확인
- 5교시: build cache와 layer 최적화 - source 변경, cache hit/miss, image size 비교
- 6교시: registry와 image provenance - Docker Hub official image, tag 전략, digest pinning, pull policy
- 7교시: tag/push/pull 흐름 - local tag, Docker Hub push는 선택, credential/secret gate 필수

- 8교시: image build failure drill - missing file, wrong CMD, wrong port, bloated context RCA

Practice Files And Assets

자료	용도
assets/day3-image-build-registry-overview.png	Day 3 image/build/registry 전체 구조 인포그래픽
labs/static-site/index.html	표준 실습 앱 HTML
labs/static-site/styles.css	표준 실습 앱 CSS
labs/static-site/Dockerfile	Dockerfile build 실습
labs/static-site/.dockerignore	build context 제외 규칙
labs/static-site/README.md	build/run/check/cleanup 예시
hands-on-lab.md	Day 3 전체 실행 흐름
assets/lesson-01-image-layers-cache-tag-digest.png	image layer/cache/tag/digest 설명
assets/lesson-02-dockerfile-instruction-map.png	Dockerfile instruction map
assets/lesson-03-source-tree-build-context.png	source tree와 build context
assets/lesson-04-build-run-pipeline.png	build/run/HTTP check pipeline
assets/lesson-05-build-troubleshooting.png	build cache/layer failure
assets/lesson-06-image-registry-provenance.png	registry와 image provenance
assets/lesson-07-tag-push-pull-gate.png	tag/push/pull security gate
assets/lesson-08-image-build-rca.png	image build failure RCA

Linux Preflight Test Evidence

Day 3 build/run 명령은 Linux 또는 macOS Docker 환경에서 사전 테스트한다. 핵심은 exact image ID가 아니라 build 성공, HTTP 확인, history/inspect evidence, cleanup이다.

항목	결과
Test OS	Ubuntu 24.04.3 LTS, Linux 6.6.87.2-microsoft-standard-WSL2, linux/amd64
Build command	<code>docker build -t paperclip-static-site:day3 .</code>
Build result	image 생성 성공, build context size 기록
Run command	<code>docker run -d --name paperclip-day3-static -p 18083:80 paperclip-static-site:day3</code>
HTTP evidence	<code>curl -I http://localhost:18083 -> HTTP/1.1 200 OK pattern</code>
History evidence	<code>docker history paperclip-static-site:day3</code> 에서 base/COPY layer 확인
Inspect evidence	image ID, repo tag, architecture, size 확인
Cache evidence	source 변경 전후 cache hit/miss 기록
Failure drill	missing file, wrong CMD, wrong port, bloated context 중 하나 재현

Required Evidence

Evidence	제출 기준
Dockerfile evidence	instruction별 역할 설명
Build evidence	build command, context size, image tag
Run evidence	container ID, <code>docker ps</code> , HTTP status
History/inspect evidence	layer, image ID, size, architecture
Cache evidence	cache hit/miss 또는 source 변경 rebuild 결과
Registry evidence	official image 기준, tag/digest 또는 push gate
Failure evidence	missing file, wrong CMD, wrong port, bloated context 중 하나 RCA
Cleanup evidence	container/image 정리 결과

End-Of-Day Checklist

- ☐ Dockerfile instruction 역할을 설명했다.
- ☐ `docker build`로 표준 앱 image를 만들었다.
- ☐ `docker run`과 `curl`로 HTTP 200을 확인했다.
- ☐ `docker history`와 `docker image inspect`를 기록했다.
- ☐ build cache 또는 source 변경 rebuild를 확인했다.
- ☐ official image tag/digest 또는 registry 출처를 확인했다.
- ☐ image build/run failure drill을 RCA 형식으로 기록했다.

- [] container/image cleanup audit을 수행했다.

Next Connection

Day 4는 runtime config와 observability/troubleshooting으로 넘어간다. Day 3에서 만든 image를 여러 환경변수와 실패 조건으로 실행하면서 logs, inspect, exec, stats를 사용해 정상/장애를 분류한다.